

Azure DevOps FFT Lab — C# Console App with YAML CI/CD Pipeline

Sommaire

1	Étape 1 — Création locale du projet C# FFT.....	1
1.1	Créer le répertoire du lab.....	1
1.2.	Créer la structure des répertoires.....	2
1.3.	Créer l'application console.....	2
1.4.	Créer le projet de tests.....	2
1.5.	Revenir à la racine.....	3
1.6.	Créer le fichier solution.....	3
1.7.	Ajouter les projets à la solution.....	3
1.8	Ajouter l'application.....	3
1.9	Ajouter les tests.....	3
1.10.	Ajouter la référence du projet principal dans les tests.....	3
1.11	Tester la compilation.....	3
1.12	Structure finale attendue.....	4
2	Étape 2 — Ajouter le vrai code FFT.....	5
3	Étape 3 — ajouter de vrais tests unitaires xUnit.....	7
4	Étape 4 — GitHub.....	8
5	Étape 5 — Création du pipeline Azure DevOps.....	10
6	Reference.....	13

1 Étape 1 — Création locale du projet C# FFT

On commence uniquement en local.
Pas encore Azure DevOps.

1.1 Créer le répertoire du lab

Ouvre un terminal PowerShell :

```
mkdir azure-devops-csharp-fft-lab  
cd azure-devops-csharp-fft-lab
```

1.2. Créer la structure des répertoires

```
mkdir src
mkdir tests
```

1.3. Créer l'application console

```
cd src
dotnet new console -n FftConsoleApp
```

```
PS C:\temp\labs\azure-devops-csharp-fft-lab\src> tree
Folder PATH listing for volume Windows
Volume serial number is 0000007A 5845:6D56
C:..
├── FftConsoleApp
│   └── obj
PS C:\temp\labs\azure-devops-csharp-fft-lab\src>
```

This PC > Windows (C:) > temp > labs > azure-devops-csharp-fft-lab > src > FftConsoleApp >				
Name	Date modified	Type	Size	
obj	5/19/2026 10:31 PM	File folder		
FftConsoleApp.csproj	5/19/2026 10:31 PM	C# Project Source ...	1 KB	
Program.cs	5/19/2026 10:31 PM	C# Source File	1 KB	

1.4. Créer le projet de tests

```
cd ..
cd tests

dotnet new xunit -n FftConsoleApp.Tests
```

```
PS C:\temp\labs\azure-devops-csharp-fft-lab\tests> tree
Folder PATH listing for volume Windows
Volume serial number is 00000052 5845:6D56
C:..
├── FftConsoleApp.Tests
│   └── obj
PS C:\temp\labs\azure-devops-csharp-fft-lab\tests>
```

1.5. Revenir à la racine

```
cd ..
```

On devrait être ici :

```
azure-devops-csharp-fft-lab
```

1.6. Créer le fichier solution

important pour Azure DevOps.

```
dotnet new sln -n FftDevOpsLab
```

1.7. Ajouter les projets à la solution

1.8 Ajouter l'application

```
dotnet sln add src/FftConsoleApp/FftConsoleApp.csproj
```

1.9 Ajouter les tests

```
dotnet sln add tests/FftConsoleApp.Tests/FftConsoleApp.Tests.csproj
```

1.10. Ajouter la référence du projet principal dans les tests

```
dotnet add tests/FftConsoleApp.Tests/FftConsoleApp.Tests.csproj reference  
src/FftConsoleApp/FftConsoleApp.csproj
```

1.11 Tester la compilation

À la racine :

```
dotnet build
```

On doit voir :

```
Build succeeded.
```

```

PS C:\temp\labs\azure-devops-csharp-fft-lab> dotnet sln add src\FftConsoleApp\FftConsoleApp.csproj
Project 'src\FftConsoleApp\FftConsoleApp.csproj' added to the solution.
PS C:\temp\labs\azure-devops-csharp-fft-lab> dotnet sln add tests\FftConsoleApp.Tests\FftConsoleApp.Tests.csproj
Project 'tests\FftConsoleApp.Tests\FftConsoleApp.Tests.csproj' added to the solution.
PS C:\temp\labs\azure-devops-csharp-fft-lab> dotnet add tests\FftConsoleApp.Tests\FftConsoleApp.Tests.csproj reference s
rc\FftConsoleApp\FftConsoleApp.csproj
Reference '..\..\src\FftConsoleApp\FftConsoleApp.csproj' added to the project.
PS C:\temp\labs\azure-devops-csharp-fft-lab>
PS C:\temp\labs\azure-devops-csharp-fft-lab> dotnet build
Determining projects to restore...
Restored C:\temp\labs\azure-devops-csharp-fft-lab\tests\FftConsoleApp.Tests\FftConsoleApp.Tests.csproj (in 265 ms).
1 of 2 projects are up-to-date for restore.
FftConsoleApp -> C:\temp\labs\azure-devops-csharp-fft-lab\src\FftConsoleApp\bin\Debug\net8.0\FftConsoleApp.dll
FftConsoleApp.Tests -> C:\temp\labs\azure-devops-csharp-fft-lab\tests\FftConsoleApp.Tests\bin\Debug\net8.0\FftConsole
App.Tests.dll
Build succeeded.
0 Warning(s)
0 Error(s)
Time Elapsed 00:00:03.03
PS C:\temp\labs\azure-devops-csharp-fft-lab>

```

1.12 Structure finale attendue

```

azure-devops-csharp-fft-lab/
├── src/
│   └── FftConsoleApp/
├── tests/
│   └── FftConsoleApp.Tests/
├── FftDevOpsLab.sln
└── README.md

```

```

henri@Henri:/mnt/c/temp/labs/azure-devops-csharp-fft-lab$ tree -L 2
.
├── FftDevOpsLab.sln
├── src
│   └── FftConsoleApp
├── tests
│   └── FftConsoleApp.Tests
└──
5 directories, 1 file

```

2 Étape 2 — Ajouter le vrai code FFT

avec :

- package NuGet MathNet.Numerics
- fichier `FFT.cs`
- calcul FFT réel
- affichage console propre

se positionner sur le répertoire source de l'application

Cd src/FftConsoleApp

Puis : installer le package suivant : MathNet.Numerics

dotnet add package MathNet.Numerics

Vérifier avec la commande suivante :

dotnet list package

```
PS C:\temp\labs\azure-devops-csharp-fft-lab\src\FftConsoleApp> dotnet list package
Project 'FftConsoleApp' has the following package references
[net8.0]:
Top-level Package      Requested  Resolved
> MathNet.Numerics      5.0.0      5.0.0

PS C:\temp\labs\azure-devops-csharp-fft-lab\src\FftConsoleApp>
```

Créer un fichier FFT.cs dans le répertoire : src/FftConsoleApp

Mettre le code suivant dedans :

```
using System.Numerics;
using MathNet.Numerics.IntegralTransforms;

namespace FftConsoleApp;

public class FFT
{
    public static Complex[] ComputeFFT(double[] signal)
    {
        Complex[] samples = signal
            .Select(x => new Complex(x, 0))
            .ToArray();

        Fourier.Forward(samples, FourierOptions.Matlab);

        return samples;
    }
}
```

Modifier program.cs

Remplacer le code existant par le code suivant :

```
using FftConsoleApp;
using System.Numerics;
```

```
double[] signal = { 1, 0, 1, 0 };

Complex[] result = FFT.ComputeFFT(signal);

Console.WriteLine("FFT Result:");

foreach (var value in result)
{
    Console.WriteLine(value);
}
```

Compiler

A la racine du projet , faire :

Dotnet build

```
PS C:\temp\labs\azure-devops-csharp-fft-lab\src\FftConsoleApp> dotnet build
Determining projects to restore...
All projects are up-to-date for restore.
FftConsoleApp -> C:\temp\labs\azure-devops-csharp-fft-lab\src\FftConsoleApp\bin\Debug\net8.0\FftConsoleApp.dll

Build succeeded.
    0 Warning(s)
    0 Error(s)

Time Elapsed 00:00:02.33
PS C:\temp\labs\azure-devops-csharp-fft-lab\src\FftConsoleApp>
```

Exécuter :

A la racine du projet , faire :

dotnet run --project src/FftConsoleApp

```
PS C:\temp\labs\azure-devops-csharp-fft-lab\src\FftConsoleApp> cd ..
PS C:\temp\labs\azure-devops-csharp-fft-lab\src> cd ..
PS C:\temp\labs\azure-devops-csharp-fft-lab> dotnet run --project src/FftConsoleApp
FFT Result:
<2; 0>
<0; 0>
<2; 0>
<0; 0>
PS C:\temp\labs\azure-devops-csharp-fft-lab>
```

3 Étape 3 — ajouter de vrais tests unitaires xUnit

Aller dans le projet de test : cd tests/FftConsoleApp.Tests

Supprimer le fichier : UnitTest1.cs

Créer le fichier : FFTTests.cs

```
using Xunit;
using System.Numerics;
using FftConsoleApp;

namespace FftConsoleApp.Tests;

public class FFTTests
{
    [Fact]
    public void ComputeFFT_ShouldReturnCorrectLength()
    {
        // Arrange
        double[] signal = { 1, 0, 1, 0 };

        // Act
        Complex[] result = FFT.ComputeFFT(signal);

        // Assert
        Assert.Equal(signal.Length, result.Length);
    }

    [Fact]
    public void ComputeFFT_ShouldNotReturnNull()
    {
        // Arrange
        double[] signal = { 1, 0, 1, 0 };

        // Act
        Complex[] result = FFT.ComputeFFT(signal);

        // Assert
        Assert.NotNull(result);
    }
}
```

Fichier FFTTests.cs crée et structure du répertoire de test :

```
PS C:\temp\labs\azure-devops-csharp-fft-lab\tests\FftConsoleApp.Tests> dir

Directory: C:\temp\labs\azure-devops-csharp-fft-lab\tests\FftConsoleApp.Tests

Mode                LastWriteTime         Length Name
----                -
d-----          5/19/2026  10:45 PM             bin
d-----          5/19/2026  10:45 PM             obj
-a----          5/19/2026  10:44 PM          788 FftConsoleApp.Tests.csproj
-a----          5/20/2026  10:54 AM          677 FFTTests.cs

PS C:\temp\labs\azure-devops-csharp-fft-lab\tests\FftConsoleApp.Tests>
```

Puis :

Revenir à la racine du projet : `cd ../../`

Executer le tests :

Dotnet test

```
PS C:\temp\labs\azure-devops-csharp-fft-lab> dotnet test
Determining projects to restore...
Restored C:\temp\labs\azure-devops-csharp-fft-lab\tests\FftConsoleApp.Tests\FftConsoleApp.Tests.csproj (in 847 ms).
1 of 2 projects are up-to-date for restore.
FftConsoleApp -> C:\temp\labs\azure-devops-csharp-fft-lab\src\FftConsoleApp\bin\Debug\net8.0\FftConsoleApp.dll
FftConsoleApp.Tests -> C:\temp\labs\azure-devops-csharp-fft-lab\tests\FftConsoleApp.Tests\bin\Debug\net8.0\FftConsoleApp.Tests.dll
Test run for C:\temp\labs\azure-devops-csharp-fft-lab\tests\FftConsoleApp.Tests\bin\Debug\net8.0\FftConsoleApp.Tests.dll
(.NETCoreApp,Version=v8.0)
VSTest version 17.11.1 (x64)

Starting test execution, please wait...
A total of 1 test files matched the specified pattern.

Passed! - Failed:    0, Passed:    2, Skipped:    0, Total:    2, Duration: 2 ms - FftConsoleApp.Tests.dll (net8.0)
PS C:\temp\labs\azure-devops-csharp-fft-lab>
```

4 Étape 4 — GitHub

Se positionner à la racine du projet : `azure-devops-csharp-fft-lab`

Initialiser git : **git init**

Créer le fichier `.gitignore` à la racine

Contenu du fichier `.gitignore`

```
bin/
obj/
.vs/
*.user
*.suo
TestResults/
```


Vérifier git : git status

Puis : **git add .**

Si on fait un git status , à nouveau, on obtient alors la figure ci-dessous : qui montre les fichiers à commiter

```
PS C:\temp\labs\azure-devops-csharp-fft-lab> git add .
PS C:\temp\labs\azure-devops-csharp-fft-lab> git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
    new file:   .gitignore
    new file:   FftDevOpsLab.sln
    new file:   src/FftConsoleApp/FFT.cs
    new file:   src/FftConsoleApp/FftConsoleApp.csproj
    new file:   src/FftConsoleApp/Program.cs
    new file:   tests/FftConsoleApp.Tests/FFTTests.cs
    new file:   tests/FftConsoleApp.Tests/FftConsoleApp.Tests.csproj

PS C:\temp\labs\azure-devops-csharp-fft-lab>
```

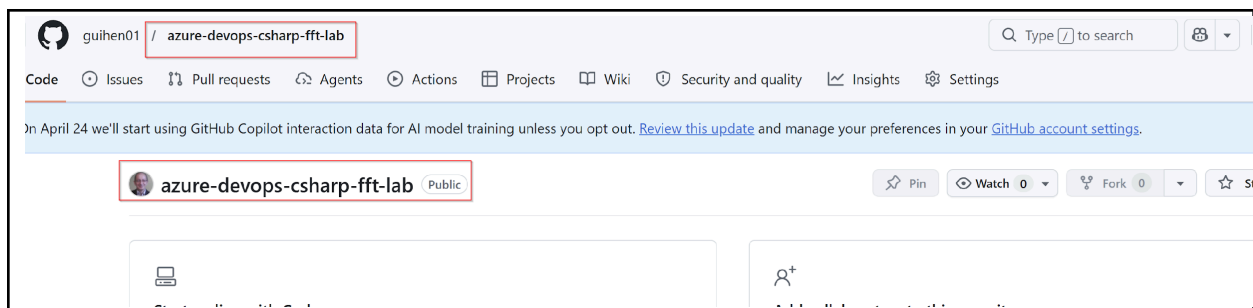
Faire le premier commit :

git commit -m "Initial commit - FFT console app with tests"

Puis créer le repo github :

Nom du repo : azure-devops-csharp-fft-lab

Le repo est créé, voir figure ci-dessous :



Ensuite exécuter la commande :

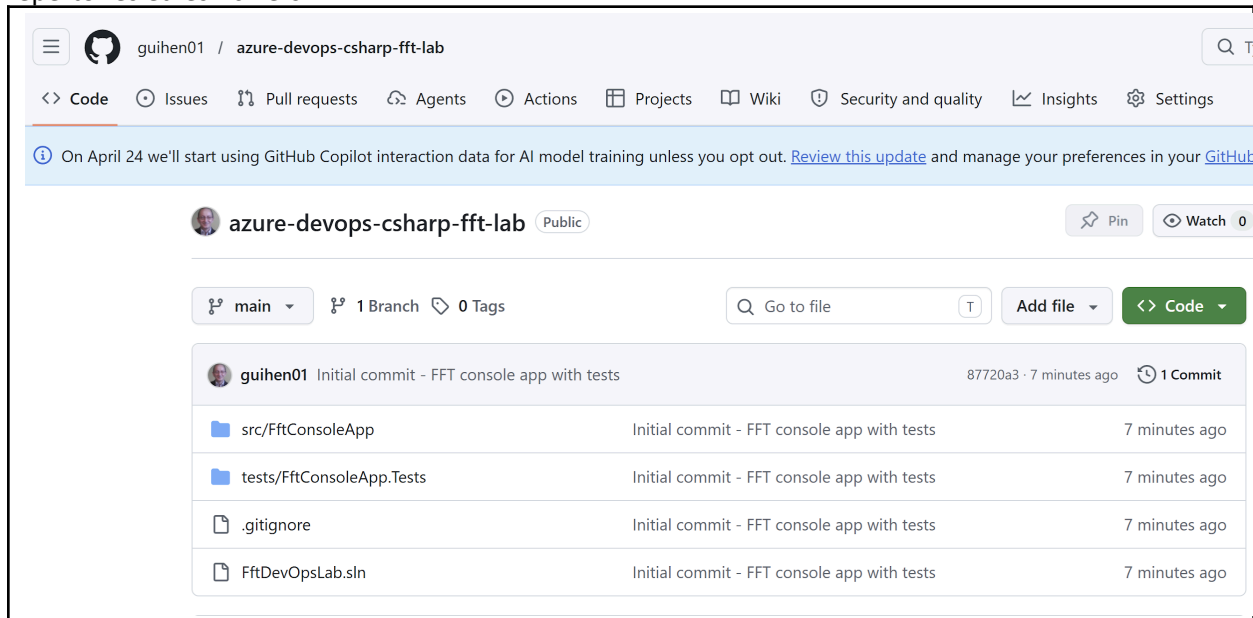
git remote add origin https://github.com/TON_USER/azure-devops-csharp-fft-lab.git

Ensuite , exécuter les commandes suivantes :

git branch -M main

git push -u origin main

On voit dans la figure ci-dessous, le repo dans github , totalement initialisé, avec la structure de répertoires et les fichiers



5 Étape 5 – Création du pipeline Azure DevOps

Aller sur Azure DevOps

<https://azure.microsoft.com/en-us/products/devops>

Créer un projet : nom du projet :

FFT-DevOps-Lab

Créer un pipeline, choisir GitHub (permissions repo)

Choisir le repo GitHub : azure-devops-csharp-fft-lab

Choisir : Existing Azure Pipelines YAML file

Revenir au niveau de powershell en local.

Dans la racine du projet local , créer le fichier : azure-pipelines.yml

```

PS C:\temp\labs\azure-devops-csharp-fft-lab>
PS C:\temp\labs\azure-devops-csharp-fft-lab> dir

Directory: C:\temp\labs\azure-devops-csharp-fft-lab

Mode                LastWriteTime         Length Name
----                -
d-----          5/19/2026 10:31 PM             src
d-----          5/19/2026 10:39 PM             tests
-a-----          5/20/2026 11:03 AM              45 .gitignore
-a-----          5/20/2026  4:05 PM             641 azure-pipelines.yml
-a-----          5/19/2026 10:44 PM            2011 FftDevOpsLab.sln

```

Mettre ce contenu : ci-dessous :

```

trigger:
- main

pool:
  vmImage: ubuntu-latest

steps:
- task: UseDotNet@2
  inputs:
    packageType: 'sdk'
    version: '8.x'

- script: dotnet restore
  displayName: Restore NuGet Packages

- script: dotnet build --configuration Release
  displayName: Build Application

- script: dotnet test --configuration Release
  displayName: Run Unit Tests

- script: dotnet publish src/FftConsoleApp -c Release -o $(Build.ArtifactStagingDirectory)
  displayName: Publish Application

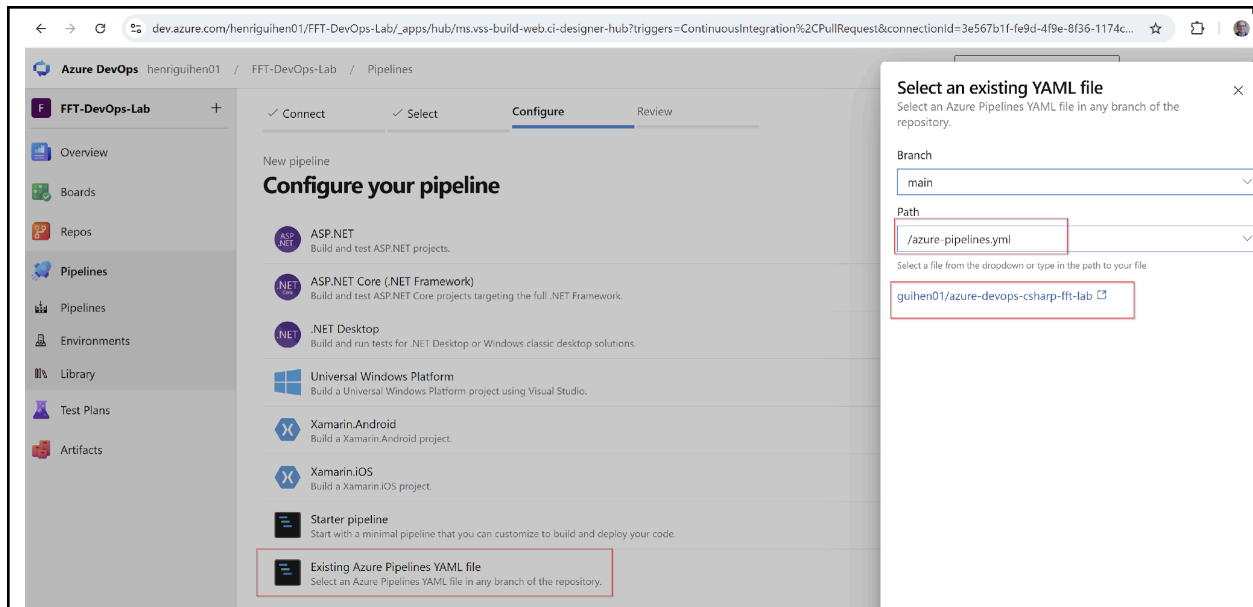
- task: PublishBuildArtifacts@1
  inputs:
    PathToPublish: '$(Build.ArtifactStagingDirectory)'
    ArtifactName: 'fft-console-app'

```

Ensuite, faire, les commandes ci-dessous :
Afin de pousser le fichier pipeline vers le repository gitHub.

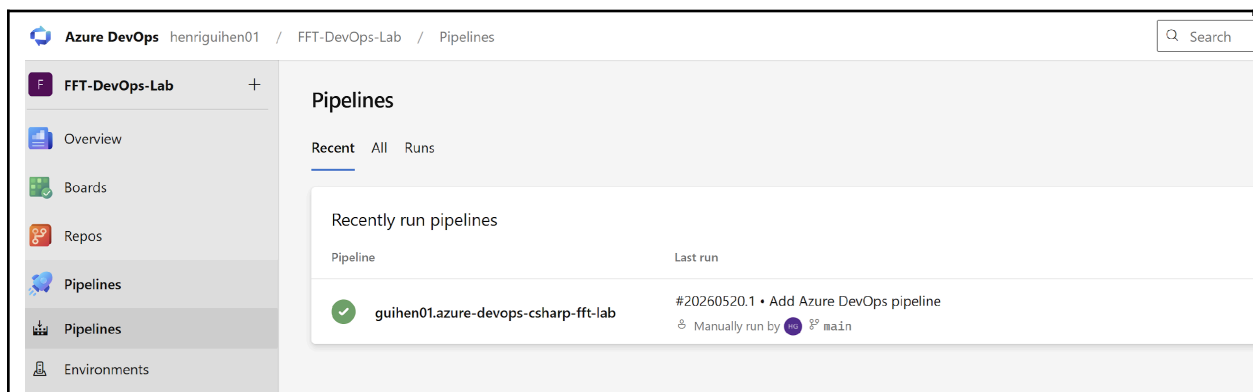
```
git add .
git commit -m "Add Azure DevOps pipeline"
git push
```

Retourner dans DevOps
Choisir : azure-pipelines.yml
Puis Run Pipeline



LE pipeline fait ensuite un : dotnet restore. Dotnet build, dotnet test, dotnet publish, crée un artifact telechargeable : fft-console-app

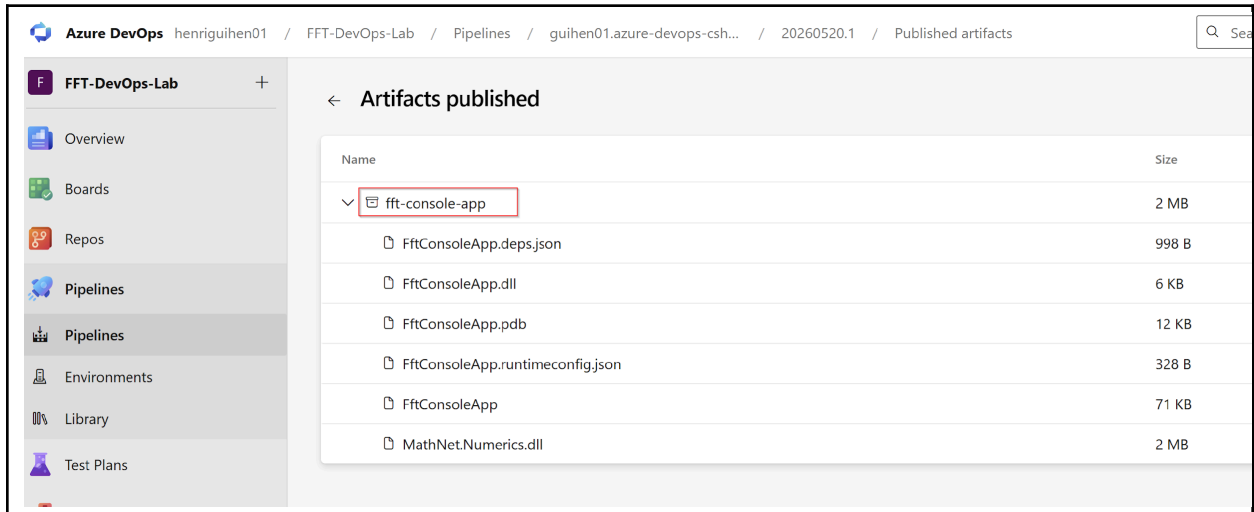
Résultat :
On voit dans la figure ci-dessous, que le pipeline (icone en vert) est fonctionnel



Si on clique sur : "

"

On devrait voir l'Artifact créée , c'est à dire le package applicatif crée par le pipeline



The screenshot shows the Azure DevOps interface for the 'FFT-DevOps-Lab' project. The left sidebar contains navigation links: Overview, Boards, Repos, Pipelines (selected), Pipelines (with a dropdown arrow), Environments, Library, and Test Plans. The main area is titled 'Artifacts published' and displays a table of artifacts. The artifact 'fft-console-app' is selected and highlighted with a red box. Below it, a list of files is shown, including 'FftConsoleApp.deps.json', 'FftConsoleApp.dll', 'FftConsoleApp.pdb', 'FftConsoleApp.runtimeconfig.json', 'FftConsoleApp', and 'MathNet.Numerics.dll'.

Name	Size
fft-console-app	2 MB
FftConsoleApp.deps.json	998 B
FftConsoleApp.dll	6 KB
FftConsoleApp.pdb	12 KB
FftConsoleApp.runtimeconfig.json	328 B
FftConsoleApp	71 KB
MathNet.Numerics.dll	2 MB

6 Reference

GitHub : <https://github.com/guihen01/azure-devops-csharp-fft-lab>